

Questões - Programação Web Avançada

Caderno de questões (sem gabarito)

Programacao Web Avancada
Universidade de Vila Velha

*Este documento foi gerado a partir do banco de questoes
estruturado no site de estudos (index.html).*

Cada questao traz o topico e a unidade correspondente.

Sumário

- Unidade 1 - Python essencial (20 questões)
- Unidade 2 - HTTP, REST e http.server (21 questões)
- Unidade 3 - Flask e primeira aplicação (11 questões)
- Unidade 4 - Application Factory e Contexto (15 questões)

Este é o caderno de questões. Marque suas respostas nas caixas (). Depois, confira no arquivo com gabarito.

Unidade 1 - Python essencial

1.

[Sintaxe básica]

Qual a diferença entre `=` e `==` em Python?

- () A) Não há diferença, são sinônimos
- () B) `=` atribui valor; `==` compara igualdade
- () C) `=` compara; `==` atribui
- () D) `==` só funciona com strings
- () E) `==` é usado apenas em funções

2.

[Estruturas de dados]

Qual estrutura remove duplicatas automaticamente e não preserva ordem?

- () A) list
- () B) tuple
- () C) set
- () D) dict
- () E) frozenset apenas

3.

[POO]

O que é um decorator em Python?

- () A) Um comentário especial
- () B) Uma função que recebe outra função e devolve uma "envelopada" com comportamento antes/depois
- () C) Um tipo de variável
- () D) Uma classe herdada
- () E) Uma sintaxe para importações

4.

[POO]

Qual mecanismo é ativado ao prefixar um atributo com dois underscores (ex: `\_\_preco`)?

- ☐ A) Encriptação automática
- ☐ B) Name Mangling (renomeia para `\_NomeClasse\_\_atributo`)
- ☐ C) Proteção absoluta contra acesso
- ☐ D) Compilação estática
- ☐ E) Cache em memória

5.

[POO]

Para que serve o decorator `@property`?

- ☐ A) Criar métodos de classe
- ☐ B) Permitir que um método seja acessado como se fosse atributo, com controle sobre leitura/escrita
- ☐ C) Herdar de outra classe
- ☐ D) Definir tipo estático
- ☐ E) Cachear o resultado

6.

[POO]

Qual argumento por convenção o `@classmethod` recebe?

- ☐ A) self
- ☐ B) cls
- ☐ C) this
- ☐ D) class
- ☐ E) me

7.

[POO]

`@staticmethod`:

- ☐ A) Recebe self
- ☐ B) Recebe cls
- ☐ C) Não recebe nem self nem cls - é essencialmente uma função dentro da classe
- ☐ D) É executado apenas uma vez
- ☐ E) É proibido em Python

8.

[POO]

O que é `super()` em Python?

- ☐ A) Cria uma nova classe
- ☐ B) Invoca métodos da classe pai a partir da filha
- ☐ C) Torna a classe mais rápida
- ☐ D) Elimina herança

☐ E) É um decorator

9.

[POO]

O que é Duck Typing?

- ☐ A) Um tipo de dado específico do Python
- ☐ B) Compatibilidade entre objetos determinada pelo comportamento (métodos disponíveis), não pela hierarquia formal
- ☐ C) Um mecanismo de compilação de tipos
- ☐ D) Uma restrição de tipos em runtime
- ☐ E) Um framework para tipagem estrita

10.

[Estruturas de dados]

Qual dessas afirmações sobre `frozenset` está CORRETA?

- ☐ A) É um dicionário congelado
- ☐ B) É um conjunto imutável, e pode ser usado como chave de dict
- ☐ C) Preserva ordem de inserção
- ☐ D) Aceita duplicatas
- ☐ E) Não existe em Python

11.

[POO]

Numa `@dataclass(frozen=True)`, tentar alterar um atributo após criação:

- ☐ A) Funciona normalmente
- ☐ B) Lança FrozenInstanceError
- ☐ C) Cria uma nova instância
- ☐ D) Ignora silenciosamente
- ☐ E) Requer recompilação

12.

[Operadores]

Qual o resultado de `10 + 3 \* 4 \*\* 2`?

- ☐ A) 208
- ☐ B) 58
- ☐ C) 148
- ☐ D) 82
- ☐ E) 100

13.

[Funções]

Uma função sem `return` explícito retorna:

- ☐ A) 0
- ☐ B) False
- ☐ C) None
- ☐ D) Lança exceção
- ☐ E) A última expressão avaliada

14.

[Operadores]

Qual o resultado de `-101 // 7` em Python?

- ☐ A) -14
- ☐ B) -15
- ☐ C) -14.4
- ☐ D) Erro
- ☐ E) 15

15.

[Type hints]

O que são type hints em Python?

- ☐ A) Restrições de tipo que causam erro em runtime se violadas
- ☐ B) Anotações opcionais úteis para legibilidade e ferramentas de análise estática
- ☐ C) Um tipo especial de decorator
- ☐ D) Um mecanismo de compilação
- ☐ E) Variáveis globais tipadas

16.

[Estrutura de arquivo]

O que o bloco `if \_\_name\_\_ == "\_\_main\_\_":` faz?

- ☐ A) Ativa modo debug do Python
- ☐ B) Executa o código apenas quando o arquivo é rodado diretamente, não quando importado
- ☐ C) Cria uma função main() automaticamente
- ☐ D) Faz o Python ignorar o arquivo
- ☐ E) É obrigatório em todo arquivo Python

17.

[Funções]

Em `def saudacao(nome, horario="dia"):` , o que é `"dia"`?

- ☐ A) O tipo esperado do parâmetro
- ☐ B) O valor padrão do parâmetro se não for passado
- ☐ C) Uma docstring
- ☐ D) Um decorator implícito

() E) Uma variável global

18.

[Funções]

O que `**kwargs` recebe em uma função?

- () A) Argumentos posicionais como tupla
- () B) Argumentos nomeados (keyword) como dicionário
- () C) Um único argumento obrigatório
- () D) Apenas strings
- () E) Argumentos padrão

19.

[Estruturas de dados]

O que caracteriza uma tupla em Python?

- () A) É mutável e ordenada
- () B) É sequência imutável, definida pela vírgula (parênteses são opcionais na maioria dos contextos)
- () C) É não ordenada
- () D) É sinônimo de lista
- () E) Só aceita tipos primitivos

20.

[Estruturas de dados]

Em Python 3.7+, dicionários preservam a ordem de:

- () A) Alfabética das chaves
- () B) Inserção
- () C) Numérica dos valores
- () D) Não preservam ordem
- () E) Aleatória

Unidade 2 - HTTP, REST e http.server

1.

[Códigos HTTP]

O que significa o código HTTP 200?

- () A) Erro do cliente
- () B) Redirecionamento
- () C) Requisição processada com sucesso (OK)
- () D) Erro do servidor
- () E) Recurso não encontrado

2.

[REST]

Qual dessas NÃO é uma restrição do REST?

- ☐ A) Cliente-Servidor
- ☐ B) Ausência de estado (stateless)
- ☐ C) Cache
- ☐ D) Interface uniforme
- ☐ E) Autenticação obrigatória

3.

[REST]

Qual restrição REST é EXPLICITAMENTE opcional?

- ☐ A) Cliente-Servidor
- ☐ B) Stateless
- ☐ C) Cache
- ☐ D) Sistema em camadas
- ☐ E) Código sob demanda

4.

[REST]

Em REST, um "recurso" é:

- ☐ A) Um registro obrigatório de banco de dados
- ☐ B) Uma abstração de informação identificável no sistema
- ☐ C) Sempre uma classe Python
- ☐ D) Um arquivo HTML
- ☐ E) Uma função remota

5.

[REST]

Qual a diferença entre recurso e representação?

- ☐ A) Não há diferença
- ☐ B) Recurso é o conceito identificado por URI; representação é a forma de transferir (JSON, HTML, XML)
- ☐ C) Recurso é HTML e representação é JSON
- ☐ D) São sinônimos
- ☐ E) Recurso é remoto, representação é local

6.

[http.server]

No módulo `http.server`, qual método é chamado automaticamente para uma requisição GET?

- ☐ A) `handle_get`
- ☐ B) `request_get`
- ☐ C) `do_GET`
- ☐ D) `on_get`

☐ E) process_get

7.

[http.server]

Qual a sequência CORRETA ao construir uma resposta HTTP no `BaseHTTPRequestHandler`?

- ☐ A) wfile.write -> send_header -> send_response
- ☐ B) send_response -> send_header -> end_headers -> wfile.write
- ☐ C) end_headers -> send_response -> wfile.write
- ☐ D) send_header -> wfile.write -> send_response
- ☐ E) Só wfile.write basta

8.

[http.server]

Qual classe do `http.server` oferece concorrência via threads?

- ☐ A) AsyncHTTPServer
- ☐ B) ThreadingHTTPServer
- ☐ C) MultiHTTPServer
- ☐ D) ConcurrentServer
- ☐ E) ForkedServer

9.

[Códigos HTTP]

O que diferencia 400 Bad Request de 404 Not Found?

- ☐ A) Nenhuma diferença
- ☐ B) 400 = a requisição em si é inválida (formato errado); 404 = a requisição é válida mas o recurso não existe
- ☐ C) 400 = servidor caiu; 404 = permissão negada
- ☐ D) Ambos indicam sucesso
- ☐ E) São ambos para redirecionamento

10.

[REST]

Uma API é RESTful quando:

- ☐ A) Só faz CRUD
- ☐ B) Segue coerentemente as restrições do estilo REST
- ☐ C) Usa HTTPS
- ☐ D) Retorna JSON
- ☐ E) É escrita em Python

11.

[HTTP]

Qual cabeçalho HTTP informa ao cliente o tipo de conteúdo do corpo?

- ☐ A) Content-Length
- ☐ B) Content-Type
- ☐ C) Content-Encoding
- ☐ D) Response-Type
- ☐ E) Body-Type

12.

[Códigos HTTP]

Código 404 significa:

- ☐ A) Erro interno do servidor
- ☐ B) Sucesso
- ☐ C) Requisição inválida
- ☐ D) Recurso não encontrado no servidor
- ☐ E) Não autorizado

13.

[WSGI]

WSGI é:

- ☐ A) Um servidor web específico
- ☐ B) Uma especificação de interface entre servidores web e aplicações Python (PEP 3333)
- ☐ C) Um framework Python
- ☐ D) Um protocolo HTTP alternativo
- ☐ E) Uma linguagem

14.

[http.server]

O módulo `http.server` é adequado para:

- ☐ A) Produção em alta escala
- ☐ B) Estudo, protótipos e testes locais
- ☐ C) Substituir Flask em produção
- ☐ D) APIs de missão crítica
- ☐ E) Sistemas com alta segurança

15.

[Códigos HTTP]

A classe 4xx representa:

- ☐ A) Sucesso
- ☐ B) Redirecionamento
- ☐ C) Erros associados à requisição do cliente
- ☐ D) Erros do servidor
- ☐ E) Respostas informativas

16.

[Códigos HTTP]

Qual código indica que a requisição foi processada e resultou em criação de novo recurso?

- ☐ A) 200 OK
- ☐ B) 201 Created
- ☐ C) 204 No Content
- ☐ D) 400 Bad Request
- ☐ E) 301 Moved

17.

[Códigos HTTP]

Código 204 significa:

- ☐ A) OK com conteúdo
- ☐ B) No Content - sucesso sem corpo de resposta
- ☐ C) Redirect
- ☐ D) Bad Request
- ☐ E) Sem sentido

18.

[Métodos HTTP]

Qual a intenção semântica de `GET`?

- ☐ A) Criar recurso
- ☐ B) Obter representação de um recurso
- ☐ C) Deletar recurso
- ☐ D) Substituir recurso
- ☐ E) Enviar formulário

19.

[Métodos HTTP]

O método `PUT`:

- ☐ A) Só obtém dados
- ☐ B) Solicita criação ou substituição da representação do recurso-alvo
- ☐ C) Só cria novo com id auto-gerado
- ☐ D) Deleta o recurso
- ☐ E) Redireciona

20.

[Métodos HTTP]

`DELETE` tem qual intenção?

- ☐ A) Criar recurso
- ☐ B) Obter recurso
- ☐ C) Substituir
- ☐ D) Solicitar remoção do recurso
- ☐ E) Autenticar

21.

[REST]

Quantas restrições arquiteturais compõem o REST?

- ☐ A) 3
- ☐ B) 4
- ☐ C) 5
- ☐ D) 6
- ☐ E) 7

Unidade 3 - Flask e primeira aplicação

1.

[venv]

Qual das alternativas melhor descreve a função de um ambiente virtual em Python?

- ☐ A) Permitir executar o Flask sem o terminal
- ☐ B) Aumentar o desempenho do interpretador
- ☐ C) Executar múltiplas aplicações Flask simultaneamente
- ☐ D) Proteger o código contra falhas de segurança
- ☐ E) Isolar dependências e evitar conflitos entre projetos

2.

[Proxies]

O que faz `current_app.config.get("CONFIG_NAME", "desconhecido")`?

- ☐ A) Cria um novo ambiente virtual
- ☐ B) Recupera o nome do ambiente de configuração ativo (com fallback)
- ☐ C) Define o nome do blueprint
- ☐ D) Obtém variável de ambiente do SO
- ☐ E) Importa o nome do pacote

3.

[Rotas]

Qual conversor de URL do Flask aceita barras "/" no valor capturado?

- ☐ A) int
- ☐ B) string
- ☐ C) float
- ☐ D) path
- ☐ E) uuid

4.

[venv]

Qual comando cria um ambiente virtual chamado `venv`?

- ☐ A) flask create venv
- ☐ B) python3 venv create
- ☐ C) python3 -m venv venv
- ☐ D) pip install venv
- ☐ E) python -m flask new-env

5.

[Application Factory]

No padrão Application Factory, qual é o papel da função `create\_app()`?

- ☐ A) Registrar as rotas diretamente em app.py
- ☐ B) Criar e retornar uma instância configurada do Flask
- ☐ C) Instalar o Flask e suas dependências
- ☐ D) Criar automaticamente um ambiente virtual
- ☐ E) Executar o servidor em produção

6.

[Variáveis de ambiente]

Qual variável de ambiente indica o ponto de entrada da aplicação Flask?

- ☐ A) FLASK_APP
- ☐ B) FLASK_DEBUG
- ☐ C) FLASK_RUN
- ☐ D) FLASK_ENTRY
- ☐ E) APP_FLASK

7.

[Blueprints]

O que são Blueprints no Flask?

- ☐ A) Um tipo especial de template HTML
- ☐ B) Sistema de cache do Flask
- ☐ C) Comando para instalar Flask
- ☐ D) Modo debug avançado
- ☐ E) Mecanismo para separar funcionalidades em módulos reutilizáveis

8.

[Flask]

Qual biblioteca abaixo NÃO é dependência principal do Flask?

- ☐ A) Werkzeug
- ☐ B) Jinja2
- ☐ C) Click
- ☐ D) SQLAlchemy
- ☐ E) ItsDangerous

9.

[Estrutura]

Em que arquivo deve ser definida a função `create_app()`, segundo a estrutura modular padrão?

- ☐ A) `app/__init__.py`
- ☐ B) `config.py`
- ☐ C) `app.py`
- ☐ D) `main/routes.py`
- ☐ E) `venv/__init__.py`

10.

[Blueprints]

Qual das alternativas corresponde a uma rota registrada corretamente em um blueprint chamado `main`?

- ☐ A) `@app.route('/')`
- ☐ B) `@route.main('/')`
- ☐ C) `@main.route('/')`
- ☐ D) `@flask.route('/')`
- ☐ E) `@Blueprint.route('/')`

11.

[venv]

Para ativar o venv no Linux/macOS:

- ☐ A) `activate venv`
- ☐ B) `venv/activate.sh`
- ☐ C) `run venv`
- ☐ D) `bash activate venv`
- ☐ E) `source venv/bin/activate`

Unidade 4 - Application Factory e Contexto

1.

[Application Factory]

Do ponto de vista arquitetural, qual é o principal objetivo do Application Factory no Flask?

- ☐ A) Melhorar a performance do servidor HTTP
- ☐ B) Evitar a criação de rotas dinâmicas
- ☐ C) Reduzir acoplamento e evitar importações circulares
- ☐ D) Substituir o uso de Blueprints
- ☐ E) Eliminar a necessidade de configuração

2.

[Contextos]

Sobre a relação entre Application Context e Request Context:

- ☐ A) Todo Application Context possui obrigatoriamente um Request Context
- ☐ B) Todo Request Context possui um Application Context associado
- ☐ C) Ambos são criados apenas na inicialização do servidor
- ☐ D) O Request Context existe antes do Application Context
- ☐ E) Eles são independentes

3.

[CLI]

Considere: `def create_app(): app = Flask(__name__); return app`. Por convenção, o que `flask run` fará ao encontrar essa estrutura?

- ☐ A) Executar automaticamente a função `create_app`
- ☐ B) Criar um servidor sem contexto
- ☐ C) Ignorar a aplicação por não existir variável global `app`
- ☐ D) Exigir obrigatoriamente um arquivo `run.py`
- ☐ E) Compilar a aplicação antes de executar

4.

[Proxies]

O objeto `g` no Flask serve para:

- ☐ A) Configurar o servidor
- ☐ B) Armazenamento temporário durante o ciclo da requisição (namespace em branco)
- ☐ C) Armazenar dados persistentes globalmente para sempre
- ☐ D) Executar tarefas cron
- ☐ E) Substituir a sessão

5.

[Exceções]

Que exceção é lançada ao acessar `current_app` sem contexto ativo?

- ☐ A) `NameError`
- ☐ B) `RuntimeError: Working outside of application context`
- ☐ C) `ImportError`
- ☐ D) `TypeError`
- ☐ E) `ValueError`

6.

[Arquitetura]

A Application Factory está associada a qual estilo arquitetural?

- ☐ A) Microserviços obrigatoriamente
- ☐ B) Serverless
- ☐ C) Monólito Modular
- ☐ D) Cliente-só

() E) P2P

7.

[GoF]

A função `create_app()` é uma aplicação prática de qual padrão do GoF?

- () A) Singleton
- () B) Observer
- () C) Factory Method
- () D) Adapter
- () E) Iterator

8.

[Import circular]

Considere: `app.py` importa `init_app` de `views.py`, e `views.py` faz `from app import app`. Qual é o problema estrutural?

- () A) Sobrecarga de memória
- () B) Falha na renderização de templates
- () C) Erro de tipagem dinâmica
- () D) Incompatibilidade com WSGI
- () E) Dependência circular entre módulos

9.

[Lazy init]

Por que dizemos que a factory usa lazy initialization?

- () A) Porque a aplicação é criada apenas quando a função é invocada
- () B) Porque o Flask executa todas as rotas antecipadamente
- () C) Porque as extensões são removidas da aplicação
- () D) Porque o Python adia a compilação do código
- () E) Porque o servidor executa em modo assíncrono

10.

[Boas práticas]

Em projetos Flask com factory, qual é considerada boa prática arquitetural?

- () A) Importar o objeto app em todos os módulos
- () B) Declarar o app como global obrigatória
- () C) Criar múltiplos objetos Flask globais
- () D) Evitar importar diretamente o objeto app nos módulos
- () E) Substituir o app por variáveis de ambiente

11.

[init_app]

Qual a principal função do padrão `init_app(app)`?

- ☐ A) Criar múltiplas instâncias do Flask
- ☐ B) Registrar componentes na aplicação de forma desacoplada
- ☐ C) Executar requisições HTTP automaticamente
- ☐ D) Substituir o uso de decorators
- ☐ E) Configurar o servidor WSGI

12.

[App Context]

O código `valor = current_app.config["DEBUG"]` só funciona corretamente quando:

- ☐ A) O servidor está em modo debug
- ☐ B) O banco de dados está conectado
- ☐ C) O Flask está rodando com Gunicorn
- ☐ D) Existe um contexto de aplicação ativo
- ☐ E) O arquivo config.py foi importado

13.

[Request Context]

Qual descreve corretamente o Request Context?

- ☐ A) Estado global permanente da aplicação
- ☐ B) Contexto criado apenas durante compilação
- ☐ C) Contexto criado a cada requisição HTTP
- ☐ D) Contexto exclusivo do banco de dados
- ☐ E) Substituto do Application Factory

14.

[Proxies]

Objetos como `request`, `current_app` e `g` são classificados como:

- ☐ A) Variáveis globais estáticas
- ☐ B) Middlewares HTTP
- ☐ C) Decoradores de rota
- ☐ D) Objetos WSGI diretos
- ☐ E) Proxies de contexto (context-local proxies)

15.

[Setup]

Durante a fase de configuração (setup) da aplicação Flask, qual ação é adequada?

- ☐ A) Acessar dados do objeto request
- ☐ B) Processar formulários HTTP
- ☐ C) Registrar blueprints e extensões
- ☐ D) Manipular sessões de usuário
- ☐ E) Executar queries de requisição ativa